WEEK 8.1 ASSIGNMENT

NAME: MOHAMMED FAISAL QURESHI

H.T NO: 2303A53015 BATCH:46

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

- Task: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.
- Requirements:
  - Password must have at least 8 characters.
  - Must include uppercase, lowercase, digit, and special character.
  - Must not contain spaces.

Example Assert Test Cases:

assert is_strong_password("Abcd@123") == True

assert is_strong_password("abcd123") == False

assert is_strong_password("ABCD@1234") == True

Expected Output #1:

Password validation logic passing all AI-generated test cases

CODE:

```
def test_password():
  assert is_strong_password("Abcd@123") == True
  assert is_strong_password("abcd123") == False
  assert is_strong_password("ABCD@1234") == False
  assert is_strong_password("Abc 123@") == False
  assert is_strong_password("Strong@1") == True
```

IMPLEMENTATION:

```
import string
def is_strong_password(password):
  if len(password) < 8 or " " in password:
    return False
  has_upper = any(c.isupper() for c in password)
  has_lower = any(c.islower() for c in password)
  has_digit = any(c.isdigit() for c in password)
  has_special = any(c in string.punctuation for c in password
  return has_upper and has_lower and has_digit and has_special
```

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

- Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.
- Requirements:
  - Classify numbers as Positive, Negative, or Zero.
  - Handle invalid inputs like strings and None.
  - Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

```
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
```

Expected Output #2:

- Classification logic passing all assert tests.

CODE:

```
def test_classify():
    assert classify_number(10) == "Positive"
    assert classify_number(-5) == "Negative"
    assert classify_number(0) == "Zero"
    assert classify_number(-1) == "Negative"
    assert classify_number(1) == "Positive"
    assert classify_number("abc") == "Invalid Input"
    assert classify_number(None) == "Invalid Input"
```

IMPLEMENTATION:

```
def classify_number(n):
    if not isinstance(n, (int, float)):
        return "Invalid Input"

    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"
```

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.
- Requirements:
  - Ignore case, spaces, and punctuation.

- Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

```
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
```

Expected Output #3:

- Function correctly identifying anagrams and passing all AI-generated tests.

CODE:

```
def test_anagram():
    assert is_anagram("listen", "silent") == True
    assert is_anagram("hello", "world") == False
    assert is_anagram("Dormitory", "Dirty Room") == True
    assert is_anagram("", "") == True
    assert is_anagram("a gentleman", "Elegant man") == True
```

IMPLEMENTATION:

```
import string

def is_anagram(str1, str2):
    def clean(s):
        return sorted(c.lower() for c in s if c.isalnum())

    return clean(str1) == clean(str2)
```

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.
- Methods:
  - add_item(name, quantity)
  - remove_item(name, quantity)
  - get_stock(name)

Example Assert Test Cases:

```
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3
```

Expected Output #4:

- Fully functional class passing all assertions.

CODE:

```
def test_inventory():
    inv = Inventory()

    inv.add_item("Pen", 10)
    assert inv.get_stock("Pen") == 10

    inv.remove_item("Pen", 5)
    assert inv.get_stock("Pen") == 5

    inv.add_item("Book", 3)
    assert inv.get_stock("Book") == 3

    inv.remove_item("Book", 5)
    assert inv.get_stock("Book") == 0

    assert inv.get_stock("Pencil") == 0
```

IMPLEMENTATION:

```
class Inventory:
    def __init__(self):
        self.stock = {}

    def add_item(self, name, quantity):
        if quantity < 0:
            return
        self.stock[name] = self.stock.get(name, 0) + quantity

    def remove_item(self, name, quantity):
        if name in self.stock:
            self.stock[name] -= quantity
            if self.stock[name] < 0:
                self.stock[name] = 0

    def get_stock(self, name):
        return self.stock.get(name, 0)
```